

Assignment 13

1. Aim

To implement and execute Factorial and Bubble Sort programs using Scala/Java.

2. Prerequisites / Setup

Step 1: Update System

```
sudo apt-get update
```

Step 2: Install Java (Required for Spark)

```
sudo apt-get install openjdk-11-jdk
```

Step 3: Download Apache Spark

```
wget https://downloads.apache.org/spark/spark-3.2.1/spark-3.2.1-bin-hadoop3.2.tgz
```

Step 4: Extract Spark

```
tar -xvzf spark-3.2.1-bin-hadoop3.2.tgz
```

Step 5: Move Spark to /opt directory

```
sudo mv spark-3.2.1-bin-hadoop3.2 /opt/spark
```

Step 6: Set Environment Variables

```
echo "export SPARK_HOME=/opt/spark" >> ~/.profile
```

```
echo "export PATH=\$PATH:\$SPARK_HOME/bin:\$SPARK_HOME/sbin" >> ~/.profile
```

```
source ~/.profile
```

Step 7: Verify Spark Installation

spark-shell

3. Program 1: Factorial

Step 1: Go to working directory

cd ~/DSBDAL_31424

Step 2: Create file

nano factorial.scala

Step 3: Write code

```
val n = 5
```

```
val nums = sc.parallelize(1 to n)
```

```
val factorial = nums.reduce((a,b) => a*b)
```

```
println("Factorial = " + factorial)
```

Step 4: Save file

CTRL + O → Enter → CTRL + X

EXECUTION

Step 1: Start Spark Shell in same directory

spark-shell

Step 2: Load file

:load factorial.scala

```
scala> :load factorial.scala
Loading factorial.scala...
n: Int = 5
nums: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at factorial.scala:24
factorial: Int = 120
Factorial = 120
```

4. Program 2: Bubble Sort

Step 1: Go to working directory

```
cd ~/DSBDAL_31424
```

Step 2: Create file

```
nano bubblesort.scala
```

Step 3: Write code

```
val arr = Array(5, 2, 8, 1, 3)
```

```
var rdd = sc.parallelize(arr)
```

```
for(i <- 0 until arr.length) {  
  rdd = rdd.mapPartitions(iter => {  
    val a = iter.toArray  
    for(j <- 0 until a.length-1) {  
      if(a(j) > a(j+1)) {  
        val temp = a(j)  
        a(j) = a(j+1)  
        a(j+1) = temp  
      }  
    }  
    a.iterator  
  })  
}
```

```
println("Sorted array:")
```

```
rdd.collect().foreach(x => print(x + " "))
```

Step 4: Save file

```
CTRL + O → Enter → CTRL + X
```

EXECUTION

Step 1: Start Spark Shell in same directory

```
spark-shell
```

Step 2: Load file

```
:load bubblesort.scala
```

```
scala> :load bubblesort.scala
Loading bubblesort.scala...
arr: Array[Int] = Array(5, 2, 8, 1, 3)
rdd: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at bubblesort.scala:24
sorted: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[5] at sortBy at bubblesort.scala:23
Sorted array:
1 2 3 5 8
```

5. Program 3: Word Count

Step 1: Go to working directory

```
cd ~/Downloads
```

Step 2: Create file

```
nano WordCount.scala
```

Step 3: Write code

```
val data = Seq("Spark is fast", "Spark is fun", "Scala is powerful")
```

```
val rdd = sc.parallelize(data)
```

```
val counts = rdd
  .flatMap(line => line.split(" "))
  .map(word => (word, 1))
  .reduceByKey(_ + _)
```

```
counts.collect().foreach(println)
```

Step 4: Save file

CTRL + O → Enter → CTRL + X

3. Execution

Step 1: Start Spark Shell in the working directory

```
spark-shell
```

Step 3: Load file

```
:load WordCount.scala
```

```
scala> :load WordCount.scala
Loading WordCount.scala...
data: Seq[String] = List(Spark is fast, Spark is fun, Scala is powerful)
rdd: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[0] at parallelize at WordCount.scala:24
counts: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[0] at parallelize at WordCount.scala:24
res0: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[1] at flatMap at WordCount.scala:24
res1: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[2] at map at WordCount.scala:24
res2: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[3] at reduceByKey at WordCount.scala:24
Spark is fast
Spark is fun
Scala is powerful
```